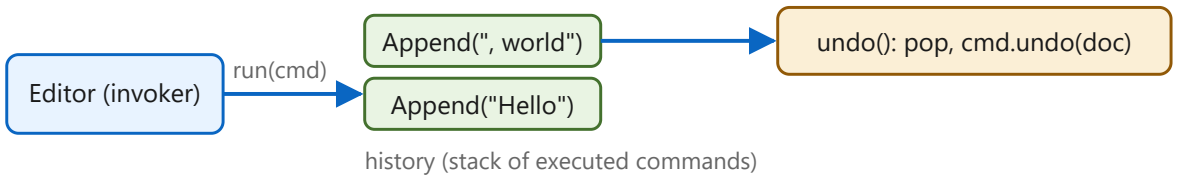
# More Patterns

Command with undo, Decorator, Adapter, Builder and Visitor with `std::variant`, with real code, plus a protocol state machine built on `variant`. Builds on Design Patterns.

## Command (with undo)

Turn an **action into an object**. Because it is an object you can store it, queue it, log it, send it, replay it, and **undo** it.




Each command knows how to execute AND how to reverse itself.

The history of command objects is the undo stack.

```
struct Command {
    virtual ~Command() = default;
    virtual void execute(Document&) = 0;
    virtual void undo(Document&) = 0;
};
struct Append : Command {
    std::string s;
    explicit Append(std::string x) : s(std::move(x)) {}
    void execute(Document& d) override { d.text += s; }
    void undo(Document& d) override { d.text.resize(d.text.size() -
s.size()); }
};
class Editor {
    Document doc_;
    std::vector<std::unique_ptr<Command>> done_;
public:
    void run(std::unique_ptr<Command> c) { c->execute(doc_);
```

```
done_.push_back(std::move(c)); }
    void undo() { if (!done_.empty()) { done_.back()->undo(doc_);
done_.pop_back(); } }
};
```

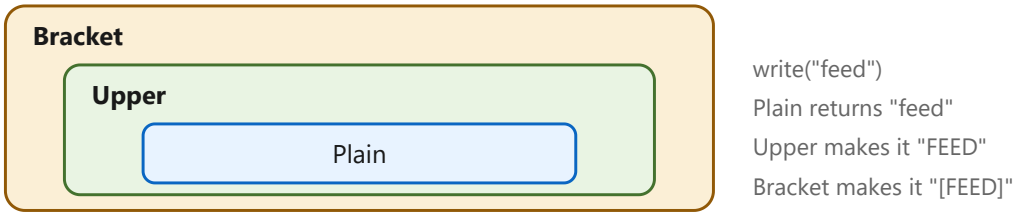
Compiled: two appends gave `"Hello, world"`, one undo gave `"Hello"`.

- **Uses:** undo/redo, task queues and thread pools (a task is a command), macro recording, transaction logs, and **order messages** that can be stored and replayed.
- A lambda stored in a `std::function` is a lightweight command when you do not need undo.
- Store enough state to reverse the action (the old value), not just the new one.

---

## Decorator

Add behaviour to an object **without changing its class** by wrapping it in another object with the **same interface**. Wrappers stack.




Each layer calls the one inside it and adds its own effect.

```
struct Sink { virtual ~Sink() = default; virtual std::string write(const
std::string&) = 0; };
struct Plain : Sink { std::string write(const std::string& s) override {
return s; } };
struct Upper : Sink {
    // wraps ANY Sink too
    std::unique_ptr<Sink> inner;
    explicit Upper(std::unique_ptr<Sink> i) : inner(std::move(i)) {}
    std::string write(const std::string& s) override {
        std::string r = inner->write(s);
        for (char& c : r) c = static_cast<char>
(std::toupper(static_cast<unsigned char>(c)));
        return r;
    }
};
```

```
struct Bracket : Sink {
    // wraps ANY Sink
    std::unique_ptr<Sink> inner;
    explicit Bracket(std::unique_ptr<Sink> i) : inner(std::move(i)) {}
    std::string write(const std::string& s) override { return "[" + inner-
>write(s) + "]"; }
};
std::unique_ptr<Sink> s = std::make_unique<Bracket>(std::make_unique<Upper>
(std::make_unique<Plain>()));
// "[FEED]" (compiled)
s->write("feed");
```

- **vs inheritance:** combinations multiply with subclasses (`UpperBracketCompressed`...), while decorators are chosen and stacked at run time.
- Seen in: stream wrappers (buffered, compressed, encrypted), logging and metrics wrappers, retry and caching layers, middleware.
- The cost is one virtual call and one allocation per layer: keep them out of hot paths.

---

## Adapter

Make an interface you **cannot change** fit the one your code **expects**. The adapter implements the expected interface and translates each call.

```
// a third-party class: cannot edit it
struct LegacySensor { int tempInTenths() const; };
// what our code uses
struct TempSource { virtual ~TempSource() = default; virtual double celsius()
const = 0; };

struct LegacyAdapter : TempSource {
    LegacySensor sensor;
    // 215 tenths -> 21.5 (compiled)
    double celsius() const override { return sensor.tempInTenths() / 10.0; }
};
```

- **Class or object adapter:** the sample holds the adaptee as a member (object adapter, preferred). Private inheritance gives a class adapter.
- A `std::function` or a lambda is often a quick adapter between two callable shapes. `std::stack` and `std::queue` are standard adapters over `deque`.

- **Adapter vs Decorator:** an adapter *changes* the interface; a decorator keeps it and adds behaviour. **Facade** hides a subsystem behind a simpler interface.

---

## Builder

Construct a complex object **step by step** with named steps, and validate at the end. Useful when a constructor would need many parameters.

```
class RequestBuilder {
    Request r_;
public:
    RequestBuilder& url(std::string u)   { r_.url = std::move(u); return
*this; }
    RequestBuilder& timeout(int secs)    { r_.timeout = secs; return *this; }
    RequestBuilder& retries(int n)       { r_.retries = n; return *this; }
    RequestBuilder& https()              { r_.secure = true; return *this; }
    Request build() const { /* check url is set, timeout > 0, ... then */
return r_; }
};
Request r =
RequestBuilder().url("example.org/api").timeout(5).retries(3).https().build()
;
```

- Each setter returns `*this` so calls chain (a **fluent interface**). Validate in `build()`: a request without a URL never gets out.
- **Alternatives in modern C++:** an aggregate with **designated initializers** (`Request{.url = "example.org", .timeout = 5}`, C++20) is shorter when no validation is needed; named parameters via a small options struct.
- The classic use is building trees or immutable objects; a builder avoids half-built objects.

## Visitor and `std::variant`

Add a new **operation** over a closed set of types without editing them, by dispatching on the type. Classic C++ does it with double dispatch (`accept(Visitor&)` and one `visit` per type); modern C++ does it with **`std::variant`** and **`std::visit`**.

```cpp
struct Circle { double r; };
struct Rect   { double w, h; };
// a closed set of alternatives, no base class, no virtuals
using Shape = std::variant<Circle, Rect>;

// one operator() per alternative
struct Area {
    double operator()(const Circle& c) const { return 3.0 * c.r * c.r; }
    double operator()(const Rect& r)   const { return r.w * r.h; }
};

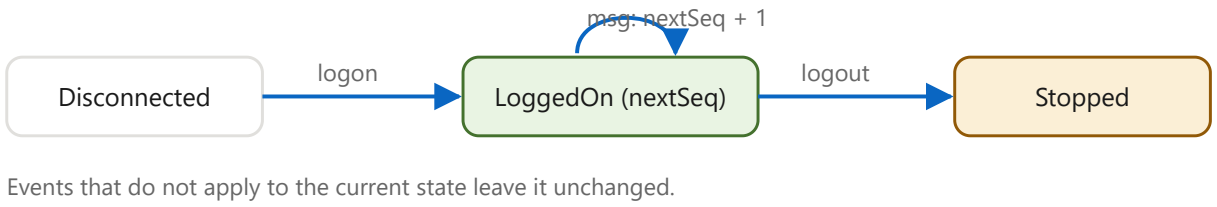
std::vector<Shape> shapes{Circle{2}, Rect{3, 4}};
double total = 0;
// 12 + 12 = 24 (compiled)
for (const auto& s : shapes) total += std::visit(Area{}, s);
```

| | VIRTUAL FUNCTIONS | `VARIANT` + `VISIT` |
|---|---|---|
| Add a new **operation** | edit every class | write one new visitor |
| Add a new **type** | write one new class | edit the variant and every visitor (the compiler reports the ones that do not handle it) |
| Storage | pointers, heap, indirect calls | values stored inline, no allocation, no virtual call |

- Use **virtual functions** when the set of types is open and operations are few; use **variant + visit** when the set of types is closed (message types, AST nodes, events) and you keep adding operations.
- A forgotten alternative is a **compile error**, not a silent bug.
- Variants are a natural fit for **protocol messages**: `std::variant<Connect, Data, Disconnect>` and a visitor that handles each kind.

## A protocol state machine with `variant`

A connection or session moves through states (disconnected, logged on, stopped) and reacts to events. Each state carries only the data it needs.




Events that do not apply to the current state leave it unchanged.

Each state carries only its own data.

```
struct Disconnected {};
struct LoggedOn { int nextSeq = 1; };
struct Stopped { std::string reason; };
using Session = std::variant<Disconnected, LoggedOn, Stopped>;

Session onEvent(const Session& s, const std::string& ev) {
    return std::visit([&](const auto& st) -> Session {
        using T = std::decay_t<decltype(st)>;
        if constexpr (std::is_same_v<T, Disconnected>) { if (ev == "logon")
return LoggedOn{}; }
        else if constexpr (std::is_same_v<T, LoggedOn>) {
            if (ev == "msg")    return LoggedOn{st.nextSeq + 1};
            if (ev == "logout") return Stopped{"logout"};
        }
        // not applicable: stay in the same state
        return st;
    }, s);
}
```

Compiled trace: `msg` while disconnected stayed **Disconnected**; after `logon` the state was **LoggedOn(next 1)**, two `msg` events moved it to **next 3**, `logout` gave **Stopped**, and a later `msg` stayed Stopped. Data such as the sequence number lives **in the state that needs it**, so it cannot be misused in another state. This is the State pattern without virtual functions (Design Patterns shows the virtual version).